

ATIMS Pipeline - Code & Implementation Architecture

Component: Non-NORAD AP taxability classification **Scope:** the batch classification pipeline delivered in this PR **Implementation:** existing ML model + Databricks Foundation Model API (`ai_classify` / `ai_query`)

1. Purpose & scope

This document describes the pipeline **as built in this PR** - the code, its structure, the data it passes between stages, and the decisions behind it. It is the implementation companion to the target architecture document (`ATIMS_Taxability_Architecture.md`), which describes the broader intended design (Vector Search and the AAW/LangGraph review agents).

The key implementation choice: **the existing trained model is preserved unchanged** (TF-IDF features + XGBoost/LightGBM ensemble + DBFS artifacts), and the LLM steps from the architecture are added through the Databricks Foundation Model API SQL functions `ai_classify()` and `ai_query()` . Vector Search is not used; the classifier keeps its existing TF-IDF features.

The pipeline runs monthly as a **9-task Databricks Workflow**. Each task is a notebook that reads a Delta staging table and writes the next, so stages are loosely coupled and individually re-runnable.

2. Repository layout

Path	Role
<code>atims_common.py</code>	Shared module: config + model layer (rules, TF-IDF, XGB/LGB, artifacts) + helpers. Imported by every task; not a workflow step.
<code>t30_ingest_prep.py</code>	Read fact table, filter to month, build <code>final_description</code>
<code>t31_rules_expert.py</code>	Rules engine -> <code>rule</code> / <code>no_rule</code>
<code>t32_classifier.py</code>	Existing ensemble + <code>ai_classify</code> + <code>ai_query</code> resolution
<code>t33_tax_lookup.py</code>	Taxability matrix join
<code>t33b_use_tax_lookup.py</code>	Reverse_UseTax logic
<code>t34_tax_writer.py</code>	<code>ai_query</code> write-ups
<code>t35_qa.py</code>	Six QA checks
<code>t36_reroute.py</code>	Escalation / finalize
<code>t37_write_output.py</code>	Prod write + MLflow metrics + Excel report + webhook
<code>atims_workflow.json</code>	Databricks Jobs spec wiring the 9 tasks
<code>docs/</code>	Architecture documents

3. Execution model

The DAG is a straight linear chain; ordering is enforced by the staging tables (a task cannot run before its input table exists).

```
t30 ingest/prep
  +-> t31 rules expert
    +-> t32 classifier
      +-> t33 tax lookup
        +-> t33b use-tax lookup
          +-> t34 tax writer
            +-> t35 QA
              +-> t36 reroute/finalize
                +-> t37 write output + notify
```

Hand-off tables (Unity Catalog Delta, defined in `atims_common.T`):

Producer	Staging table
t30	<code>t30_prepped</code>
t31	<code>t31_rules_labeled</code>
t32	<code>t32_classified</code>
t33	<code>t33_tax_lookup</code>
t33b	<code>t33b_use_tax</code>
t34	<code>t34_tax_writer</code>
t35	<code>t35_qa</code>
t36	<code>t36_rerouted</code>
t37	<code>PROD_TABLE</code> (final)

Tasks read/write via the helpers `ac.read_stage()` / `ac.write_stage()` , which keeps the I/O contract in one place.

4. Shared module (`atims_common.py`)

Holds three things:

1. **Configuration** - catalog/schema, table names, the LLM endpoint, the `ai_classify` label set, optional reference tables, the webhook, and the DBFS model paths. (See Section 10.)
2. **The model layer, ported verbatim** from the prior production notebook: the rules engine (`build_labels` , `_apply_*_rules`), the feature builder (`build_feature_matrix` - TF-IDF + categoricals + vendor hash + log amount), the artifact loader (`load_artifacts`), and the ensemble predictor (`predict_no_rule`).
3. **Helpers** - Spark/dbutils accessors, Delta read/write, `ai_classify` label array / prompt-label builders, and the legacy cluster-discovery + Excel report used by t37.

The model code is unchanged; only the post-prediction routing was moved out of `predict_no_rule` into t32 so the architecture's agreement logic lives there.

5. Task-by-task implementation

t30 - Ingest & prep

Reads `VMAP_FACT_TABLE` (Delta), filters to `ACCOUNTING_DATE LIKE '<month>%'`, and builds `final_description` by coalescing `LINE_DESCRIPTION` / `DESCRIPTION` / `DIST_ALL_DESCRIPTION` with filler values stripped. No model.

t31 - Rules expert

Applies the three rule tiers in order - keyword, generic, Parker-Volpe - each overwriting the previous (later match wins). Rule-matched rows receive a `cluster` and `label_account_code` (confidence implied 1.0) and `label_source = 'rule'`; everything else is `label_source = 'no_rule'`. The architecture's Vector-Search fallback is intentionally not used; this is deterministic-rules-only, matching the legacy behavior.

t32 - Classifier (core)

Follows the architecture's two-sub-model design:

- **Sub-model A** is the existing ensemble, loaded from DBFS artifacts and run on the cleaned text plus label-encoded categoricals (`EXTC_CODE` , `MIC` , `COMPANY_CODE` , `STATE` , `INVOICE_SOURCE`), a hashed vendor feature, and a `log1p` amount. It outputs an **account code** (`final_pred`) and `final_conf`.
- **Sub-model B** is `ai_classify(final_description, array(<CATEGORY_LABELS>))`, which outputs a **category name** (`llm_pred`).

Because A speaks account codes and B speaks category names, the optional `ACCOUNT_TO_CATEGORY` map puts A into category space (`model_category`).

Agreement logic:

Condition	Outcome	Tier / path
agree, <code>final_conf > 0.8</code>	accept	HIGH / STANDARD
agree, lower conf	accept	MEDIUM / STANDARD
disagree	<code>ai_query('databricks-gpt-4o', ...)</code> resolves to one category	RESOLVED / COMPLEX

The ML predictions are joined back at **line grain** (`invoice_id + inv_line_number`) - this fixes a legacy bug where the join keyed on `invoice_id` alone and fanned out multi-line invoices.

t33 - Tax lookup

`LEFT JOIN` to the taxability matrix on `final_category + state -> taxability` (Taxable | Exempt | null) and `tax_source` (`matrix` | `no_match`). A null taxability sets `review_priority_pre = HIGH` (no Tax-Reasoning agent; unmatched goes straight to human review).

t33b - Use-tax lookup

Pure `withColumn CASE` on `use_tax_amount + taxability`: `>0 & Exempt -> 'Yes'`; `>0 & Taxable -> 'No'`; `=0 -> 'No'`; `use_tax NULL` or `taxability NULL -> NULL` (deferred branch, pending client confirmation of NULL semantics).

t34 - Tax writer

`ai_query('databricks-gpt-4o', ...)` generates `full_writeup` from the structured fields; when `Reverse_UseTax = 'Yes'` the prompt forces the write-up to state the reversal requirement and reference the amount. `short_justification` is a lightweight template (avoids a second LLM call), `audit_ready = taxability not null`. The architecture's prior-write-up RAG is omitted (needs Vector Search).

t35 - QA

Six Spark window/statistical checks: vendor/category consistency, amount z-score, state/category history (optional table), rule/classifier conflict (dormant - the classifier doesn't run on rule rows), taxability sanity (optional table), and batch-level outlier. Produces `anomalies[]`, `anomaly_score`, `review_priority`, `override_suggested`, `recommendation`. `review_priority` respects the HIGH set in t33 (it is never downgraded).

t36 - Re-route / finalize

The architecture re-ran the classifier with a QA hint via the LLM; with a deterministic ensemble that re-run is a no-op, so this task **finalizes** instead: `override_suggested` rows are escalated to `need_to_review`, `processing_path` is labeled (FAST / STANDARD / COMPLEX / ESCALATION), and an `iteration` column is kept for schema parity.

t37 - Write output + notify

Selects the final schema, appends to `PROD_TABLE` partitioned by `ACCOUNTING_DATE`, logs batch metrics to the existing MLflow experiment, generates the legacy need-to-review Excel report (with TF-IDF KMeans cluster discovery), and POSTs the completion webhook to the FastAPI backend.

6. Model layer (unchanged)

Element	Detail
Text features	TF-IDF, 10,000 features, word uni+bigrams, sublinear TF, min_df=2
Categoricals	LabelEncoder on EXTC/MIC/COMPANY_CODE/STATE/INVOICE_SOURCE (unseen -> UNKNOWN)
Vendor	MD5 hash -> 5,000 bins
Amount	log1p(abs(TRAN_AMOUNT))
Classifier	XGBoost (hist) + LightGBM, probabilities averaged 50/50
Target	label_account_code
Artifacts	DBFS: xgb_model.ubj , lgb_model.txt , label_encoder.pkl , cat_encoders.pkl , tfidf.pkl
Tracking	MLflow experiment ap_tax_classification (per-run batch metrics)

The pipeline **loads** these artifacts; it does not retrain. Retraining remains owned by the existing training process.

7. LLM integration

- **ai_classify** (t32) - zero-shot category from CATEGORY_LABELS .
- **ai_query** (t32 resolution, t34 writer) - GPT-4o via LLM_ENDPOINT .
- **Labels** - CATEGORY_LABELS in atims_common.py (replace the seed list with the full category set).
- **Cost lever** - ACCOUNT_TO_CATEGORY . Without it, the ensemble (account codes) and ai_classify (category names) disagree on most rows, so nearly everything falls to ai_query resolution. Wiring the map brings resolution back to the intended ~10-15% of unmatched rows.

8. Output schema (PROD_TABLE)

Append-mode, partitioned by ACCOUNTING_DATE :

```
invoice_id, inv_line_number, final_description, vendor_name, state, tran_amount,
final_category, taxability, tax_source, use_tax_amount, Reverse_UseTax,
final_pred, final_conf, rule_matched, decision_path,
xgb_pred, xgb_conf, lgb_pred, lgb_conf, models_agree, confidence_tier,
short_justification, full_writeup, audit_ready,
anomalies, anomaly_score, review_priority, override_suggested, recommendation,
iteration, processing_path, label_source, accounting_date
```

9. Configuration reference (atims_common.py)

Key	Purpose
CATALOG , STAGING_SCHEMA , PROD_TABLE	Unity Catalog targets
LLM_ENDPOINT	Foundation Model endpoint for ai_query (default databricks-gpt-4o)
CATEGORY_LABELS	label list for ai_classify (populate with full category set)
ACCOUNT_TO_CATEGORY	account_code -> category map (cost / agreement lever)
TAXABILITY_MATRIX	category + state -> taxability reference
USE_TAX_COL	source use-tax column name
HISTORICAL_STATS , STATE_RULES	optional QA reference tables
WEBHOOK_URL	FastAPI completion endpoint
PATHS	source table + DBFS model artifact paths

10. Deployment & run order

1. Place all files in one Databricks Repo / Git folder so import atims_common resolves as a sibling module.
2. Edit the config block in atims_common.py (Section 9).
3. Import atims_workflow.json (Workflows -> Create -> JSON) and set existing_cluster_id on each task. The cluster must have the Foundation Model API (ai_classify / ai_query) enabled.
4. Run with parameter invoice_month=YYYY-MM . Order is fixed: t30 -> t31 -> t32 -> t33 -> t33b -> t34 -> t35 -> t36 -> t37 .

11. Design decisions & known limitations

- **Existing model preserved.** The existing TF-IDF feature pipeline is kept. The only structural change vs. the prior notebook is the line-grain join fix in t32.
- **Vector Search omitted.** The Rules-Expert similarity fallback and the Tax-Writer RAG both rely on Vector Search; ai_classify / ai_query are wired in their place. RAG can be added to t34 later without touching other tasks.
- **Account-code / category space.** Set ACCOUNT_TO_CATEGORY to keep ai_query cost in the intended range (see Section 7).

- **QA Check 4 dormant.** The classifier runs only on no_rule rows, so rule rows have no model prediction to compare against. Enable a shadow prediction pass in t32 to activate it.
- **Use-tax NULL deferred.** Reverse_UseTax is NULL when use_tax_amount is NULL, pending confirmation of NULL-vs-data-gap semantics.
- **Train/serve population.** The model was trained on rule-matched rows but runs on no_rule rows; reported training accuracy reflects in-rule performance, not the served population.

12. Implemented vs target architecture

Target architecture element	This PR
9-task Databricks Workflow, Delta hand-offs	Implemented
Rules Expert (deterministic)	Implemented
Classifier ensemble + ai_classify + ai_query resolution	Implemented (Sub-model A = existing ensemble)
Tax Lookup / Use-Tax / Tax Writer / QA / Re-route / Output	Implemented
Vector Search (rules fallback, write-up RAG)	Omitted
XGBoost serving endpoint	Not used - artifacts loaded from DBFS
AAW/LangGraph review & feedback agents (Phases 4-5)	Out of scope - consumes PROD_TABLE + webhook downstream